

✓ 5.1 합성곱 신경망

합성곱 신경망(CNN): 이미지의 국소적 부분을 계산함으로써 시간과 자원을 절약하여 이미지의 세밀한 부분까지 분석할 수 있는 신경망

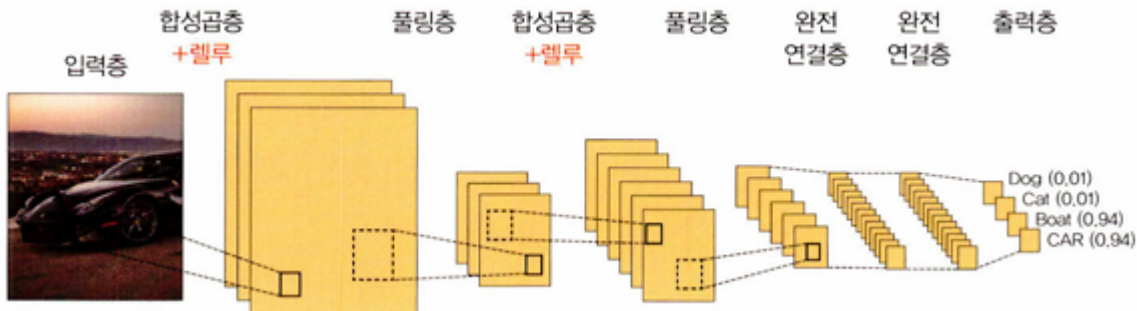
합성곱층의 필요성

- 이미지나 영상을 처리하는 데 유용
- 이미지 분석은 $n \times n$ 배열을 펼쳐서 각 픽셀에 가중치를 곱하여 은닉층으로 전달, 데이터의 공간적 구조를 무시 - 합성곱층이 이를 방지

합성곱 신경망 구조

- 입력층: 입력 이미지 데이터가 최초로 거치게 되는 계층
 - 이미지: 단순 1차원 데이터가 아닌 높이, 너비, 채널의 값을 가짐
 - 채널은 이미지가 그레이스케일이면 1, 컬러이면 3
- 합성곱층: 입력 데이터에서 특성을 추출하는 역할
 - 특성 추출: 입력 이미지가 들어왔을 때 이미지에 대한 특성을 감지하기 위해 커널이나 필터 사용
 - 커널/필터는 모든 영역을 훑으면서 특성 추출, 추출된 것이 특성 맵
 - 커널은 3×3 , 5×5 크기로 적용되는 것이 일반적, 스트라이드라는 지정된 간격에 따라 순차적으로 이동
 - 커널은 스트라이드 간격만큼 순회하면서 모든 입력 값과 합성곱 연산으로 새로운 특성 맵을 만들
 - 컬러 이미지일 경우 채널이 3이고, RGB 각각에 서로 다른 가중치로 합성곱을 적용
 - 필터가 두 개 이상이면 필터 각각이 특성 추출 결과의 채널이 됨
 - 입력 데이터: $W * H * D$ (W: 가로, H: 세로, D: 채널 또는 깊이)
 - 하이퍼파라미터: 필터 개수(K), 필터 크기(F), 스트라이드(S), 패딩(P)
- 풀링층: 특성 맵의 차원을 다운 샘플링(이미지를 축소)하여 연산량을 감소, 주요한 특성 벡터 추출
 - 풀링 연산: 최대 풀링(최댓값 추출), 평균 풀링(평균 반환)
 - 대부분의 합성곱 신경망에서는 최대 풀링 사용
 - 평균 풀링은 각 커널 값을 평균화시켜 중요한 가중치를 갖는 값의 특성이 희미해질 수도
 - 입력 데이터: $W * H * D$ (W: 가로, H: 세로, D: 채널 또는 깊이)

- 하이퍼파라미터: 필터 크기(F), 스트라이드(S)
- 완전연결층: 차원 축소된 특성 맵이 최종적으로 전달
 - 이미지는 3차원 벡터에서 1차원 벡터로 펼쳐짐
- 출력층: 소프트맥스 활성화 함수 사용, 입력받은 값을 0~1 사이의 값으로 출력
 - 이미지가 각 레이블에 속할 확률 값이 출력, 가장 높은 확률 값을 갖는 레이블이 최종 값



1D, 2D, 3D 합성곱

- 1D 합성곱: 필터가 시간을 축으로 좌우로만 이동할 수 있는 합성곱
 - 입력(W)과 필터(k)에 대한 출력은 W
- 2D 합성곱: 방향 두 개로 움직이는 형태
 - 입력(W,H)과 필터(k,k)에 대한 출력은 (W,H)
- 3D 합성곱: 필터가 움직이는 방향이 3개
 - 입력(W,H,L)과 필터(k,k,d)에 대한 출력은 (W,H,L)
 - $d < L$ 을 유지하는 것이 중요

3D 입력을 갖는 2D 합성곱: 입력이 3D 형태임에도 출력 형태가 2D 행렬

- 필터에 대한 길이가 입력 채널의 길이와 같아야 해서 이런 형태가 만들어짐
- ex) LeNet-5, VGG

1X1 합성곱: 3D 형태로 입력

- 채널 수를 조정해서 연산량이 감소되는 효과
- ex) GoogLeNet

✓ 5.2 합성곱 신경망 맛보기

파이토치는 기본적으로 GPU 사용을 권장하나, GPU가 장착되지 않은 환경에서도 파이토치 정상 사용 가능

torchvision.datasets: 다양한 데이터셋 포함

- 주요 파라미터

- root: 데이터셋을 내려받을 위치 지정
- download: True면 해당 위치에 해당 데이터셋이 있는지 확인한 후 내려 받음
- transform: 이미지를 텐서(0~1)로 변경

torch.utils.data.DataLoader: 원하는 크기의 배치 단위로 데이터를 불러오거나 순서가 무작위로 섞이도록 함

- 주요 파라미터

- 데이터를 불러올 데이터셋 지정
- batch_size: 데이터를 배치로 묶어 줌

+) 객체 지향 프로그래밍: 프로그래밍에서 필요한 데이터를 추상화하여 속성이나 행동, 동작, 특징 등을 객체로 만들고, 그 객체들이 서로 유기적으로 동작하도록 하는 프로그래밍 방법

- 객체: 메모리를 할당받아 프로그램에서 사용되는 모든 데이터를 의미

+) 클래스와 함수

- 함수: 하나의 특정 작업을 수행하기 위해 독립적으로 설계된 프로그램 코드
- 클래스: 함수 뿐만 아니라 관련 변수까지도 한번에 묶어서 관리하고 재사용할 수 있게 해주는 것

nn.Linear: 단순 선형 회귀 모델을 만들 때 사용

- 파라미터

- in_features: 입력의 크기
- out_features: 출력의 크기

torch.nn.Dropout(p): p만큼의 비율로 텐서의 값이 0이 되고, 0이 되지 않는 값들은 기존 값에 $(1/(1-p))$ 만큼 곱해져 커짐

forward(): 모델이 학습 데이터를 입력 받아서 순전파 학습을 진행

view: 텐서의 크기를 변경

nn.xx와 nn.functional.xx 비교

구분	nn.xx	nn.functional.xx
형태	nn.Conv2d: 클래스 nn.Module 클래스를 상속받아 사용	nn.functional.conv2d: 함수 def function (input)으로 정의된 순수한 함수
호출 방법	먼저 하이퍼파라미터를 전달한 후 함수 호출을 통해 데이터 전달	함수를 호출할 때 하이퍼파라미터, 데이터 전달
위치	nn.Sequential 내에 위치	nn.Sequential에 위치할 수 없음
파라미터	파라미터를 새로 정의할 필요 없음	가중치를 수동으로 전달해야 할 때마다 자체 가중치를 정의

Autograd: 자동 미분을 수행하는 파이토치의 핵심 패키지

- 자동 미분에 대한 값을 저장하기 위해 테이프를 사용

분류 문제에서 클래스가 세 개 이상일 때 주의할 점

- 정확도가 80% 이상: 모든 클래스가 동등하게 고려된 것인지, 특정 클래스의 분류가 높았던 것인지에 유의
- 정확도가 90% 이상: 100개 중 90개가 하나의 클래스에 속할 경우 정확도가 높다고 볼 수 없음

nn.Sequential: 계층을 차례로 쌓을 수 있도록 $Wx+b$ 와 같은 수식과 활성화 함수를 연결해 주는 역할

nn.Conv2d()

- 파라미터
 - in_channels: 입력 채널의 수
 - out_channels: 출력 채널의 수
 - kernel_size: 커널의 크기
 - padding: 패딩 크기
- 출력 크기 구하는 공식: $(W-F+2P)/S+1$

BatchNorm2d: 학습 과정에서 각 배치 단위별로 데이터가 다양한 분포를 가지더라도 평균과 분산을 이용하여 정규화하는 것

MaxPool2d: 이미지 크기를 축소시키는 용도로 사용

- 파라미터
 - kernel_size: mXn 행렬로 구성된 가중치
 - stride: 입력 데이터에 커널을 적용할 때 이동할 간격
- 출력 크기 구하는 공식: IF/F

